

FootballGPS 機能仕様書

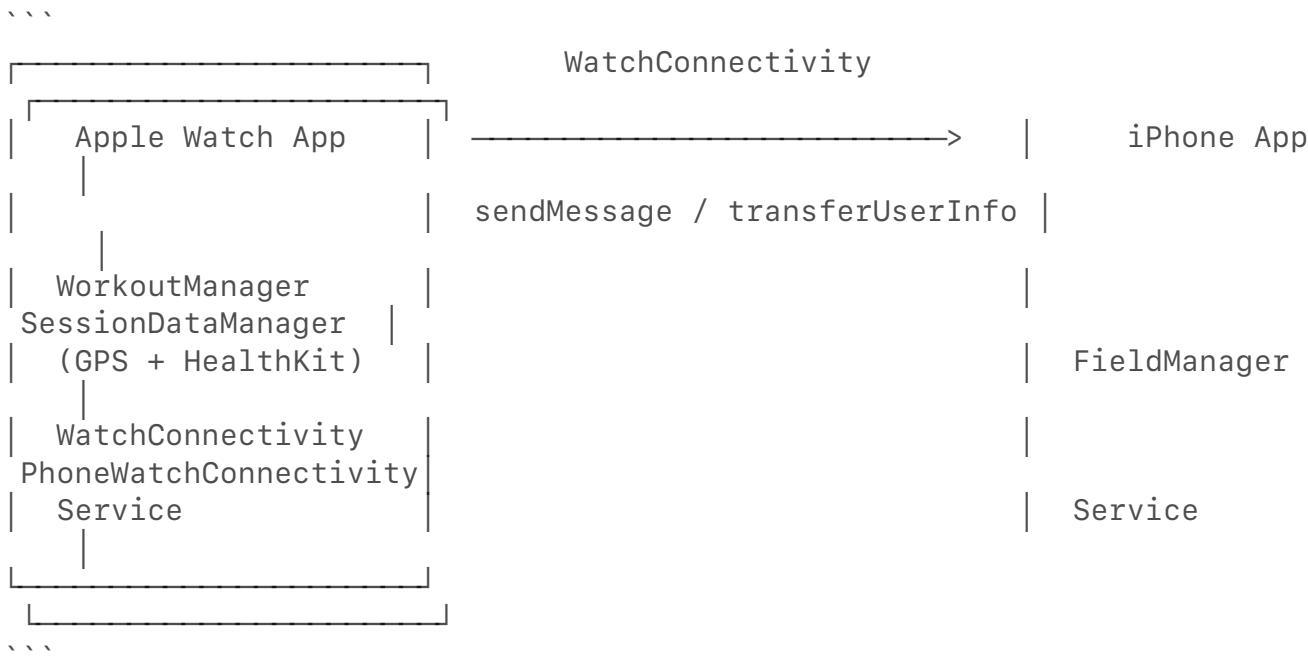
- > 最終更新: 2026-05-02
- > 対象バージョン: 1.0.0
- > 作成元: ソースコード全体の静的解析

1. アプリ概要

****FootballGPS**** は、Apple Watch で GPS
ベースのサッカートレーニングデータを記録し、iPhone で可視化・分析するアプリ。

項目	内容
プラットフォーム	iOS (iPhone) + watchOS (Apple Watch)
アーキテクチャ	SwiftUI + HealthKit + CoreLocation + WatchConnectivity
データ永続化	UserDefaults (JSON エンコーディング)
認証/バックエンド	未実装 (Firestore 対応の dictionary プロパティのみ準備済み)

2. システム構成図



3. データモデル

3.1 TrainingSession

****ファイル:**** `ModelsTrainingSession.swift`

プロパティ	型	説明
`id`	`String`	UUID ベースの一意識別子
`userId`	`String`	ユーザー ID (現状は固定値 `temp_user_id`)
`teamId`	`String?`	所属チーム ID
`fieldId`	`String?`	紐付けフィールド ID
`name`	`String`	セッション名 (デフォルト: `yyyy/MM/dd HH:mm`)
`date`	`Date`	開始日時
`duration`	`TimeInterval`	セッション時間 (秒)
`visibility`	`SessionVisibility`	公開設定 (`public` / `private` / `team_admin_only`)
`totalDistance`	`Double`	総走行距離 (メートル)
`maxSpeed`	`Double`	最高速度 (m/s)
`avgSpeed`	`Double`	平均速度 (m/s)
`sprintCount`	`Int?`	スプリント回数 (現状は未計算、フィールドのみ存在)
`gpsDataPath`	`String?`	Cloud Storage パス (未使用、将来の Firestore 連携用)

3.2 GPSPoint / GPSPData

****ファイル:**** `ModelsTrainingSession.swift`

プロパティ	型	説明
`timestamp`	`Date`	記録時刻
`latitude` / `longitude`	`Double`	GPS 座標
`speed`	`Double`	速度 (m/s)。`CLLocation.speed` の負値は 0 に補正
`altitude`	`Double`	高度 (メートル)
`horizontalAccuracy`	`Double`	水平精度 (メートル)

`GPSPData` は `sessionId` と `[GPSPoint]` の配列をまとめた JSON 保存用コンテナ。

3.3 Field

****ファイル:**** `ModelsField.swift`

プロパティ	型	説明
`corners`	`FieldCorners`	フィールド 4 隅の GPS 座標 (topLeft/topRight/bottomRight/bottomLeft)
`dimensions`	`FieldDimensions`	実寸 (`length` x `width` メートル)。標準値: 105m x 68m

****主要ロジック:****

- **convertToFieldCoordinate(_):** GPS 座標をフィールドローカル座標 (0~width, 0~length) に変換。平行四辺形座標系で連立方程式を解く。フラットアース近似 (`metersPerLat=110540``, `metersPerLon=111320*cos(lat)``) を使用。
- **contains(coordinate:)** Ray Casting Algorithm でフィールド内外判定。
- **matchRate(for:)** GPSPoint 配列に対するフィールド内一致率を返す。
- **correctToRectangle(...)** (static): ユーザーが指定した 4 点を最小二乗フィットで直角長方形に補正。重心・長辺方向の平均・直交する短辺方向を計算。

3.4 Team / UserProfile

ファイル: `ModelsTeam.swift``

- `Team``: 招待コード (6桁、紛らわしい文字除外)、管理者/メンバー権限管理。
- `UserProfile``: Firebase Auth UID ベース。
- **現状:** UI はプレースホルダーのみ。「ステップ3で実装予定」と明記。

4. 計測ロジック (Apple Watch)

4.1 WorkoutManager

ファイル: `WatchWorkoutManager.swift``

4.1.1 HealthKit 連携

項目	詳細
ワークアウトタイプ	<code>`.soccer`</code> / <code>`.outdoor`</code>
共有権限	<code>HKWorkoutType`</code>
読取権限	<code>heartRate`</code> , <code>activeEnergyBurned`</code> , <code>distanceWalkingRunning`</code> , <code>workoutRoute`</code>
セッション管理	<code>HKWorkoutSession`</code> + <code>HKLiveWorkoutBuilder`</code>

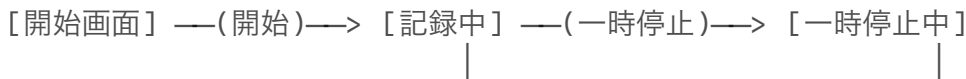
> **注意:** `HKLiveWorkoutBuilderDelegate.didCollectDataOf`` のコールバック内でデータ抽出処理は未実装。心拍数・歩数・消費カロリーは権限要求のみで、実際の値は取得していない。

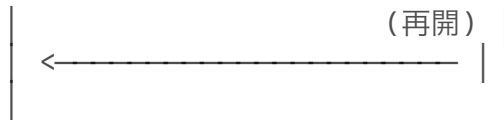
4.1.2 GPS 記録ロジック

...

CLLocationManager 設定:

- `desiredAccuracy: kCLLocationAccuracyBest`
- `distanceFilter: 2m` (2メートル移動ごとに更新)
- `activityType: .fitness`
- `allowsBackgroundLocationUpdates`: コメントアウト (要実機設定)





(終了確認) → [サマリー画面] → (完了) → [開始画面]

...

状態遷移	isRunning	isPaused	GPS記録	タイマー	HKSession
開始	`true`	`false`	`startUpdatingLocation`	動作中	`.running`
一時停止	`true`	`true`	`stopUpdatingLocation`	停止	`.paused`
再開	`true`	`false`	`startUpdatingLocation`	動作中	`.running`
終了	`true`*	`false`	`stopUpdatingLocation`	停止	`.ended`
サマリー閉じる	`false`	-	-	-	-

> *終了時に `isRunning` を即座に `false` にしない設計。サマリー画面表示のため `showingSummary` で制御し、ユーザーが「完了」ボタンを押した時点で `isRunning = false` + `reset()`。

5.2 バックグラウンド動作

項目	現状
`allowsBackgroundLocationUpdates`	**コメントアウト** (実機テスト時に有効化が必要)
Info.plist `UIBackgroundModes`	未設定 (要: `location`, `workout-processing`)
HKWorkoutSession	実機ではバックグラウンド継続が可能 (watchOS の仕様)

> **課題:** 現在のコードではバックグラウンドでの GPS 記録が無効。実機で試合中に手首を下げると記録が途切れる可能性がある。

5.3 経過時間の計測

- `Timer.scheduledTimer` で 1 秒間隔カウントアップ
- 一時停止時にタイマーを `invalidate`、再開時に再作成
- **一時停止中は `elapsedTime` が加算されない** (実効プレイ時間のみ計測)

6. データ転送 (Watch -> iPhone)

6.1 転送フロー

...

```
[Watch] endWorkout()
```

```

|
|— 実機: WatchConnectivityService.sendSession()
|   |— isReachable ? sendMessage (即時) : transferUserInfo
|       (キューイング)
|       |— sendMessage 失敗時 -> transferUserInfo にフォールバック
|— シミュレーター: UserDefaults に直接保存
    |— pendingSession_{id}, pendingGPS_{id}, pendingSessionIds
...

```

6.2 iPhone 側受信

****`PhoneWatchConnectivityService`**** が以下を処理:

1. `didReceiveMessage` (リアルタイム) / `didReceiveUserInfo` (キューイング) で受信
2. `type == "trainingSession"` を検証
3. `TrainingSession` + `GPSData` を辞書からパース
4. ****フィールド自動判定****: `FieldManager.detectField(for:)` で GPS ポイントの 80% 以上がフィールド内にあれば紐付け
5. `SessionDataManager.saveSession()` で永続化

7. データ永続化

7.1 SessionDataManager

****ファイル:**** `SessionDataManager.swift`

キー	内容
`savedSessions`	`[TrainingSession]` の JSON (ISO 8601 日付)
`gpsData_{sessionId}`	`GPSData` の JSON

- シングルトン (`@MainActor`)
- メモリキャッシュ (`gpsDataCache`) + UserDefaults 二層構造
- セッションは日付降順ソート

7.2 FieldManager

****ファイル:**** `FieldManager.swift`

キー	内容
`savedFields`	`[Field]` の JSON
`selectedFieldId`	現在選択中のフィールド ID

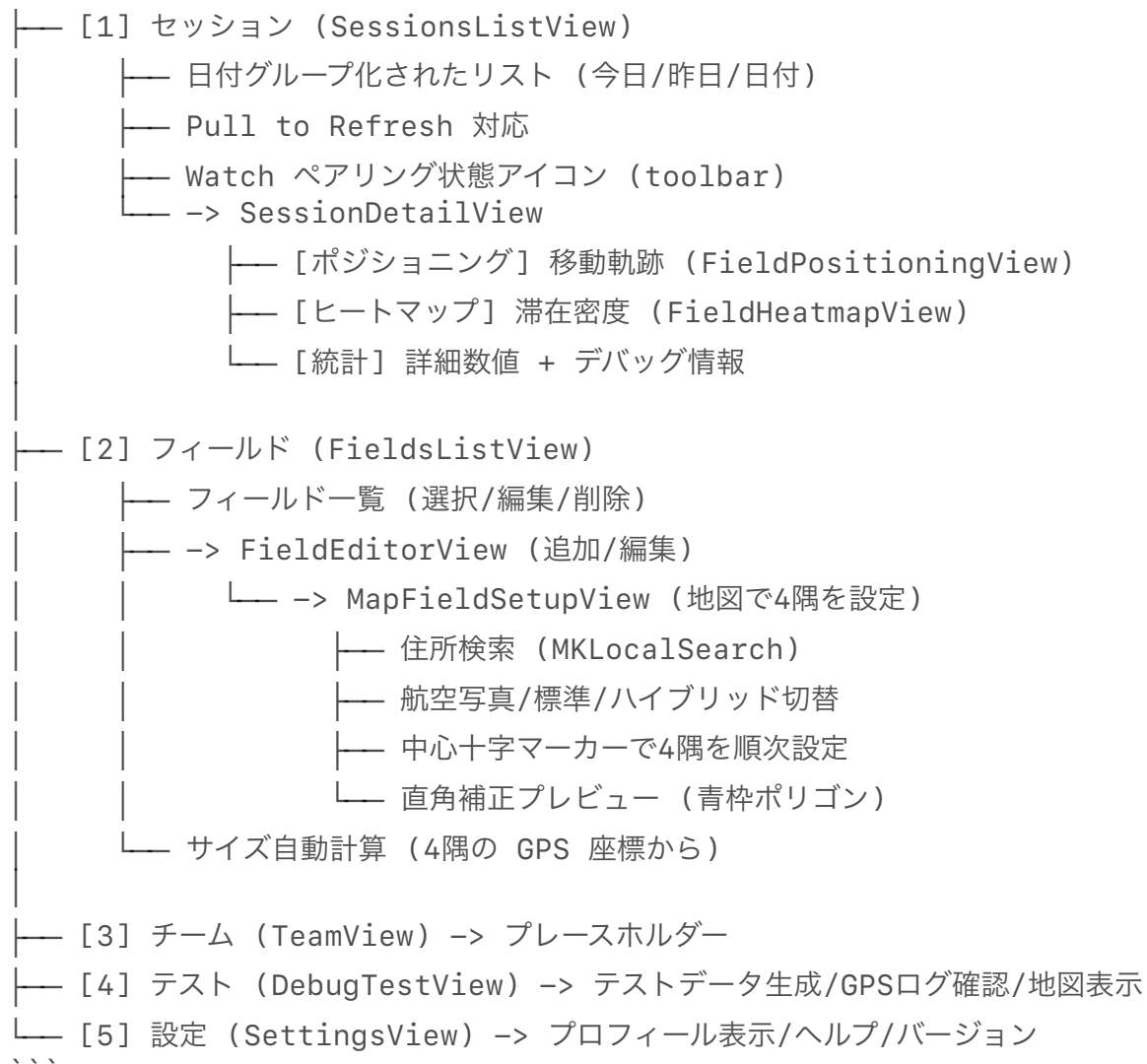
- 最初のフィールド追加時に自動選択
- `detectField(for:)`: 全登録フィールドに対して `matchRate` を計算し、80%以上で最高一致率のものを返す

8. UI/UX 設計

8.1 iPhone アプリ (5タブ構成)

...

TabView



...

8.2 Apple Watch アプリ

...

WorkoutView



```

└─ [記録中] RunningView
    │
    │   └─ 経過時間（大きなフォント、緑色）
    │   └─ リアルタイム統計：距離 / 速度 / 最高速度 / GPS点数
    │   └─ 「GPS詳細」ボタン -> GPSDebugView（最新20点の座標/速度/精度）
    │   └─ [一時停止/再開] トグルボタン（左）
    │   └─ [終了] ボタン（右、確認ダイアログ付き）
    └─ [サマリー] SummaryView
        │
        │   └─ 最終統計：時間/距離/平均速度/最高速度/GPS点数
        │   └─ iPhone 送信状態表示
        │   └─ 「完了」ボタン -> リセットして開始画面へ
        ...

```

8.3 試合中操作の設計意図

```

| 設計判断 | 意図 |
|-----|-----|
| Watch で記録、iPhone で分析 | 試合中は Watch の最小操作のみ。詳細分析は試合後に iPhone で |
| 大きな経過時間表示 | グランスで時間確認可能 |
| 一時停止/終了の 2 ボタン分離 | 誤操作防止。終了は確認ダイアログ付き |
| 終了後もサマリー表示を維持 | ユーザーが結果を確認してから明示的に閉じる |
| 自動フィールド判定 | 選手がフィールドを手動選択する手間を省略 |
| GPS 精度フィルタ（50m） | 屋内/トンネル等の低精度データを除外 |
| `distanceFilter: 2m` | 静止時のノイズ軽減と移動検出の精度バランス |

```

9. ビジュアライゼーション

9.1 FieldPositioningView（移動軌跡）

****ファイル:**** `ViewsFieldPositioningView.swift`

- GPS ポイントをフィールド座標に変換し、`Path` で線描画
- 開始地点：緑丸 + "START"、終了地点：赤丸 + "END"
- フィールド背景：緑色 +
白ラインで外枠/センターライン/センターサークル/ペナルティエリア/ゴールエリアを描画
- アスペクト比維持（width/length）

9.2 FieldHeatmapView（滞在密度）

****ファイル:**** `ViewsFieldHeatmapView.swift`

- グリッドサイズ：20列 x 14行（長さ方向 x 幅方向）

- GPS ポイントをフィールド座標に変換 -> グリッドインデックスに配置
- 正規化（最大値 = 1.0）後、4段階グラデーション：
 - `0~0.25`：青系（低密度）
 - `0.25~0.5`：緑系
 - `0.5~0.75`：黄系
 - `0.75~1.0`：赤系（高密度）
- 90度回転表示（ゴールが上下）

9.3 FieldOverlayView（デバッグ地図オーバーレイ）

****ファイル:**** `ContentView.swift`（DebugTestView 内）

- GPS 座標を画面座標に変換し、Canvas でフィールド枠を描画
- 4隅にカラーマーカー（緑/青/赤/橙）
- 赤色点線枠 + 半透明塗りつぶし

10. 未実装機能・課題一覧

#	カテゴリ	内容	関連コード
1	**計測**	スプリント回数の自動検出（速度閾値ベース）	`sprintCount` フィールド存在、ロジック未実装
2	**計測**	HealthKit からの心拍数取得	`heartRate` プロパティ存在、`didCollectDataOf` 未実装
3	**計測**	HealthKit からの歩数/カロリー取得	権限要求のみ
4	**バックグラウンド**	`allowsBackgroundLocationUpdates` の有効化	コメントアウト状態
5	**バックグラウンド**	Info.plist の `UIBackgroundModes` 設定	未設定
6	**認証**	Firebase Auth / ユーザー認証	`userId` は固定値 `temp_user_id`
7	**バックエンド**	Firestore 連携（セッション/フィールドの同期）	`dictionary` プロパティのみ準備済み
8	**チーム**	チーム機能（招待/メンバー管理/データ共有）	モデルは実装済み、UI はプレースホルダー
9	**フィールド**	`MapFieldSetupView` の現在地取得	TODO コメント、デフォルト東京座標
10	**分析**	ゾーン別滞在時間（攻撃/中盤/守備エリア）	未実装
11	**分析**	速度ゾーン別走行距離（歩行/ジョグ/ランニング/スプリント）	未実装
12	**分析**	セッション間の比較・トレンド表示	未実装
13	**UI**	デバッグタブの条件付き表示（`#if DEBUG`）	現在は常時表示

| 14 | ****データ**** | UserDefaults から Core Data / SwiftData への移行 |
大量データ時のパフォーマンス懸念 |

11. ファイル構成マップ

iPhone App (`FootballGPS/`)

ファイル	責務
`FootballGPSApp.swift`	エントリポイント。WatchConnectivity 初期化、シミュレーター用テストデータ投入
`ContentView.swift`	TabView ルート + DebugTestView + GPSTestMapView + FieldOverlayView
`ModelsTrainingSession.swift`	TrainingSession / GPSPoint / GPSTestData モデル
`ModelsField.swift`	Field モデル (座標変換、内外判定、直角補正)
`ModelsTeam.swift`	Team / UserProfile モデル
`SessionDataManager.swift`	セッション CRUD + UserDefaults 永続化
`FieldManager.swift`	フィールド CRUD + 自動判定 + UserDefaults 永続化
`PhoneWatchConnectivityService.swift`	Watch からのデータ受信 + パース + フィールド自動判定
`MockData.swift`	モックデータ (セッション/フィールド/チーム/ユーザー)
`ViewsSessionsListView.swift`	セッション一覧 (日付グループ、Pull to Refresh)
`ViewsSessionDetailView.swift`	セッション詳細 (ポジショニング/ヒートマップ/統計タブ)
`ViewsFieldsListView.swift`	フィールド一覧 (選択/編集/削除)
`ViewsFieldEditorView.swift`	フィールド追加/編集フォーム
`ViewsMapFieldSetupView.swift`	地図で4隅設定 (MKMapView + 検索 + 直角補正)
`ViewsFieldPositioningView.swift`	移動軌跡描画 + FieldBackgroundView + FieldLinesView
`ViewsFieldHeatmapView.swift`	ヒートマップ描画
`ViewsTeamView.swift`	チーム画面 (プレースホルダー)
`ViewsSettingsView.swift`	設定画面

Watch App (`FootballGPS Watch App/`)

ファイル	責務
`WatchFootballGPS_Watch_App.swift`	エントリポイント。WatchConnectivity 初期化

`WatchWorkoutManager.swift`	ワークアウト管理 (HealthKit + GPS + タイマー + シミュレーター対応)
`WatchWorkoutView.swift`	全 Watch UI (StartView / RunningView / SummaryView / GPSDebugView / TestDataSendView)
`WatchConnectivityService.swift`	iPhone へのデータ送信 (リアルタイム + キューイング)
`ContentView.swift`	未使用 (デフォルトテンプレート)

12. 設計上の前提・制約

1. ****単一ユーザー前提****: 認証未実装のため `userId` は固定値。マルチユーザー対応は Firebase Auth 導入後。
2. ****オフライン優先****: データは端末ローカル (UserDefaults) に保存。クラウド同期は未実装。
3. ****GPS 精度依存****:
フィールド座標変換はフラットアース近似。赤道付近や極地では誤差が大きくなるが、サッカーフィールド規模では問題なし。
4. ****Watch -> iPhone 一方向****: データは Watch で記録 -> iPhone に転送の一方向フロー。iPhone から Watch への指示送信は未実装。
5. ****フィールド登録が前提****: ヒートマップ・ポジショニング表示にはフィールドの 4 隅 GPS 座標登録が必須。未登録時は案内画面を表示。